

Objectives:

- Threads
- Synchronization

Learning Material:

Threads => <https://www.cs.cmu.edu/afs/cs/academic/class/15492-f07/www/pthreads.html>
<https://w3.cs.jmu.edu/kirkpams/OpenCSF/Books/csf/html/ThreadArgs.html>

Exercise: 01

Write a program where two threads attempt to withdraw money from a shared bank account [balance amount in a shared memory], you can use sleep in withdraw function.

- Show how simultaneous withdrawals can lead to incorrect balance calculations.
- Use Synchronization (mutex) to fix the issue (race condition)

Exercise: 02

Write a C program where 3 threads print their thread number 3 times each. Without mutex the output will be mixed and jumbled. Add a mutex so that each thread completes all 3 of its print statements before the next thread starts printing.

Expected output pattern:

```
Thread 1 printing: 1
Thread 1 printing: 2
Thread 1 printing: 3
Thread 2 printing: 1
Thread 2 printing: 2
Thread 2 printing: 3
.....
```

Exercise: 03

A library has 3 computers available. 7 students (threads) want to use a computer. Each student uses the computer for 2 seconds (sleep(2)). Write a C program using a counting semaphore initialized to 3 so that:

- At most 3 students are using computers at any time
- Each student prints when they sit down and when they leave
- All 7 students eventually get their turn

Exercise: 04

Write a C program with 2 threads:

- Thread A prints "Data Produced"
- Thread B prints "Data Consumed"

Without synchronization, either can print first. Use semaphores to guarantee that "Data Produced" always prints before "Data Consumed" no matter how many times the loop runs. The loop should repeat 5 times.

Expected output always:

```
Data Produced
Data Consumed
Data Produced
Data Consumed
.....
```

Exercise: 05

Thread	Main()
<pre>void * (void * p) { int * n = new int(*(int*)p); While(1) { cout << "I am in " << *n << endl; cout << "I am out" << *n << endl; } }</pre>	<pre>{ For(I = 0 ; I < 5 ; i++) { Create thread and send value of i } }</pre>

Synchronize the above program using semaphores in such a way that your output becomes:

- I am in 1
- I am in 2
- I am in 3
- I am in 4
- I am in 5
- I am out 1
- I am out 2
- I am out 3
- I am out 4
- I am out 5
- I am in 1
- I am in 2
- I am in 3
- I am in 4
- I am in 5
- I am out 1
- I am out 2
-

The Pattern will repeat and will never end. Try using as few semaphores as possible.

